

MEMORY MANAGEMENT

Condensed Study Notes

Generated: 2026-05-25 00:04 UTC

Memory is a computer's storage area for instructions and data needed by applications and the operating system.

Key memory operations:

- `read(address)`: Retrieves content from a specific memory location.
- `write(address, value)`: Stores a value into a specific memory location.

The Memory Manager is the part of the operating system responsible for allocating memory to processes. When demand exceeds available memory, the system swaps content between memory and storage. Different situations require different memory management techniques.

Memory is a critical resource, alongside the CPU, as processes require it for their code, stack, heap (for dynamic structures), and data (variables). Processors fetch instructions and access data from memory.

LOGICAL VERSUS PHYSICAL ADDRESSES IN AN OPERATING SYSTEM

When a program runs, the CPU generates a logical address. This is also known as a virtual address because it doesn't exist physically. It's how the CPU refers to a location.

A physical address, on the other hand, pinpoints the exact physical spot where data is stored in memory.

A hardware component called the Memory-Management Unit (MMU) is responsible for translating logical addresses into their corresponding physical addresses.

Users and their programs work with logical addresses, believing they are running in their own address space. However, for execution, these logical addresses must be mapped to physical memory locations by the MMU before use.

The term physical address space refers to all the physical addresses that correspond to the logical addresses within a logical address space.

Differences between Logical and Physical Address in Operating System

Logical addresses are generated by the CPU from the program's perspective, while physical addresses represent actual locations in memory.

- **Logical Address Space**: The set of all logical addresses generated by the CPU for a program.
- **Physical Address Space**: The set of all physical memory locations mapped to corresponding logical addresses.

A logical address does not physically exist in memory, whereas a physical address is a directly accessible memory location.

- Logical addresses are generated by the CPU.
- Physical addresses are computed by the Memory Management Unit (MMU).

Compile-time and load-time address binding methods generate identical logical addresses. This differs from run-time address binding.

Mapping Virtual Addresses to Physical Addresses

Address binding is the process of mapping one address space to another. Users interact only with logical (virtual) addresses, which the CPU generates. The Memory Management Unit (MMU) translates these logical addresses into physical addresses, representing locations in RAM.

Address binding can occur at three different times:

- **Compile Time:** If the process's memory location is known during compilation, an absolute physical address is generated directly in the executable. Loading is fast, but if the address is occupied, the program crashes, requiring recompilation.
- **Load Time:** If the memory location isn't known at compile time, relocatable addresses are generated. The loader translates these to absolute addresses by adding the process's base address in memory to each logical address. If the base address changes, the process must be reloaded.
- **Execution Time:** This is used when a process can be moved between memory locations during execution (e.g., dynamic linking, compaction). Memory can be allocated or deallocated while the process is running.

Comparison Chart

This chart compares Logical Addresses and Physical Addresses across several parameters:

- **Basic Generation:**
 - Logical Address: Generated by the CPU.
 - Physical Address: The actual location in memory.
- **Address Space:**
 - Logical Address Space: The set of all logical addresses generated by a program.
 - Physical Address Space: The set of all physical addresses mapped to the logical addresses.
- **Visibility:**
 - Logical Address: The user can view the logical address of a program.
 - Physical Address: The user can never view the physical address of a program.
- **Generation:**
 - Logical Address: Generated by the CPU.
 - Physical Address: Computed by the Memory Management Unit (MMU).
- **Access:**
 - Logical Address: The user can use the logical address to access the physical address.
 - Physical Address: The user can indirectly access the physical address, but not directly.

SWAPPING IN OPERATING SYSTEM

Swapping is a memory management technique to improve main memory utilization by temporarily moving processes between main memory (RAM) and secondary storage (hard disk).

- **Swap-out:** Moving a process from RAM to the hard disk.
- **Swap-in:** Moving a process from the hard disk back into RAM.

This is used when data needed by an application is not currently in RAM. The space on the hard disk where swapped-out processes are stored is called **swap space**.

While swapping can affect system performance, it enables running larger programs and multiple processes simultaneously. It is sometimes referred to as memory compaction.

Example Calculation of Transfer Time:

- Process size: 2048 KB
- Data transfer rate: 1 Mbps = 1024 kbps
- Time = Process size / Transfer rate = 2048 KB / 1024 kbps = 2 seconds = 2000 milliseconds

Total time including both swap-in and swap-out is 4000 milliseconds.

Memory Management Unit (MMU)

The Memory Management Unit (MMU) is a hardware device responsible for the runtime mapping between virtual (logical) addresses and physical addresses.

The Operating System, in its role of memory management, handles processes by moving them between disk and main memory for execution. It also tracks which parts of memory are in use and which are available.

Advantages of Swapping

Swapping offers several key benefits for memory management:

- **Enhanced CPU Multitasking:** Swapping allows the CPU to handle multiple tasks concurrently. This means processes don't experience lengthy waits for execution, leading to smoother multitasking.
- **Improved Main Memory Utilization:** By moving processes between RAM and secondary storage, swapping frees up valuable main memory. This enables larger programs to run and increases overall memory efficiency.
- **Virtual Memory Creation:** Swapping is fundamental to the creation and operation of virtual memory systems. This allows programs to use more memory than physically available.
- **Process Management:** It assists the CPU in managing numerous processes within a single main memory space effectively.

Disadvantages of Swapping

Swapping, while beneficial for memory management, has drawbacks:

1. **Data Loss Risk:** If the computer loses power, any program information that has been substantially swapped out to secondary storage might be lost.
2. **Performance Degradation:** An inefficient swapping algorithm can lead to more "Page Faults" (requests for data not currently in main memory), which in turn slows down overall system performance.

MemoryAllocation

Memory allocation strategies determine how a process is assigned memory space.

There are two main approaches:

- **Contiguous Memory Allocation:** A process is allocated a single, continuous block of memory.
- This is often achieved by dividing memory into fixed-sized partitions.
- **Non-Contiguous Memory Allocation:** A process can occupy multiple memory blocks located at different addresses.
- This allows a process to acquire memory segments as needed, potentially scattered throughout the available memory.

In contiguous allocation, a single section of a contiguous memory block is given to a process based on its requirements.

MemoryPartitioning

Memory partitioning is a method for contiguous memory allocation, dividing memory into distinct sections.

Two main types of partitioning exist:

- **Fixed Partitioning (Static Partitioning):**
 - Memory is divided into a set of non-overlapping partitions of fixed, unmovable sizes.
 - Each partition typically holds one process.
 - Limits multiprogramming, as the total number of processes is determined by the number of partitions.
- **Variable Partitioning (Dynamic Partitioning):**
 - Memory is divided into non-overlapping partitions of variable sizes.
 - Partition sizes are not declared initially but are determined dynamically based on the size of the process being loaded.
 - This system aims to prevent internal fragmentation by making partition sizes match process sizes.

Note

- In a single-tasking OS, only one process uses the program area of memory and remains there until completion.

- In a multitasking OS, if active processes exceed main memory capacity, a process is swapped out to make room for others.

Pros of Contiguous Memory Allocation

This method allows a user to access files randomly.

It provides excellent read performance.

Implementation is fairly simple.

Cons of Contiguous Memory Allocation

Contiguous memory allocation can present challenges.

One difficulty is that a file may become larger than its initially allocated space, making it hard to accommodate growth.

Another issue is disk fragmentation, where free memory space becomes broken into small, non-contiguous pieces, making it hard to allocate larger blocks even if total free space is sufficient.

Partition Allocation Methods

These are four common memory management techniques that use different placement algorithms to allocate processes to memory partitions.

- **FirstFit:** Scans memory from the beginning and allocates the first available block (hole) that is large enough for the process.
- **BestFit:** Scans the entire list of free holes and allocates the smallest hole that is greater than or equal to the process size.
- **WorstFit:** Scans the entire list of free holes and allocates the largest hole available. This is the opposite of BestFit.
- **NextFit:** Similar to FirstFit, but it starts searching from the last allocation point and allocates the first sufficient partition it finds.

Paging in Operating System

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory.

Key Concepts:

- **Pages:** Fixed-size blocks that a process's logical address space is divided into.
- **Page Frames:** Fixed-size blocks that main memory is divided into. Page frames are the same size as pages.
- **Paging:** The process of retrieving pages from secondary storage into main memory.

How it works:

1. A process is separated into pages.
2. Main memory is split into page frames.
3. When a process needs memory, the OS allocates one or more page frames and maps the process's logical pages to these physical page frames.
4. The mapping is maintained by a **pagetable**, used by the Memory Management Unit (MMU) to translate logical addresses to physical addresses.
5. The pagetable maps each logical page number to a physical page frame number.

Characteristics:

- Allows the physical address space of a process to be non-contiguous.
- Page sizes and frame sizes must be the same.
- Process pages are typically stored in secondary storage until needed in main memory.

Example of Paging in OS

Paging allows a process to be saved in multiple, non-contiguous memory frames. Imagine main memory as a set of 1KB frames. If main memory is 16KB, there are 16 frames.

Processes are divided into 1KB pages. For example, four processes (P1, P2, P3, P4), each 4KB in size, would require 4 pages each. Initially, all frames are empty, allowing pages to be stored contiguously.

When processes like P2 and P4 move to a waiting state, their frames (8 in total) become free. These freed frames can then be used to load other pages. For instance, an 8KB process (P5, which has 8 pages) from the ready queue can be loaded into the available frames.

SEGMENTATION IN OPERATING SYSTEM

Segmentation divides a process into chunks called segments. These segments are not necessarily of the same size.

Segmentation provides a user's view of a process, which paging does not. This user view is then mapped to physical memory.

Types of Segmentation in Operating System

Segmentation involves mapping logical addresses to physical addresses. A Segment Table stores information about these segments.

Two types of segmentation are:

- **Simple Segmentation:** Each process is divided into segments. All segments are loaded into memory at runtime, but not necessarily in contiguous blocks.

- **Virtual Memory Segmentation:** Processes are divided into segments, but this segmentation doesn't necessarily happen all at once. Segmentation may or may not occur during program runtime.

What is Segment Table

The Segment Table translates a two-dimensional logical address into a one-dimensional physical address. Each entry in the table contains:

- **Base Address:** The starting physical address where a segment is located in memory.
- **Segment Limit (also Segment Offset):** The length of the segment.

A logical address is composed of a segment number (s) and a segment offset (d). The segment number identifies which segment the address belongs to, and the segment offset indicates the specific location within that segment. The segment offset (d) requires a certain number of bits to represent the size of the segment.

Advantages of Segmentation in Operating System

Segmentation offers several advantages in operating systems:

- **Sharing:** Allows memory segments to be shared between processes. This is useful for inter-process communication or sharing code libraries. Users specify segment sizes, unlike paging where hardware determines page size.
- **No Internal Fragmentation:** Unlike fixed-size partitions, segmentation avoids wasted space within allocated memory blocks.
- **Security & Protection:** Segments can be used to segregate data from operations, and protection exists between segments, preventing one process from accessing or modifying another's memory. This enhances system security and stability.
- **Improved CPU Utilization:** Loading a complete module at once improves CPU utilization.
- **Flexibility:** Provides greater flexibility than paging. Segments can be variable-sized, and processes can have multiple segments for more fine-grained memory allocation.
- **Space Efficiency:** The Segment Table consumes less space compared to a Page Table in paging.

Segmentation allows user programs to be divided into modules, which are essentially separate processes' codes.

Disadvantages of Segmentation in Operating System

Segmentation can lead to increased access times because it requires two memory accesses: one for the segment table and another for the main memory to retrieve the instruction.

Overhead is introduced by the segment table itself, as each entry requires additional memory. Accessing this table to find memory locations can slow down memory operations. Keeping a segment table for each activity adds to this overhead.

Fragmentation: Similar to contiguous allocation, segmentation can suffer from external fragmentation. As processes are loaded and removed, free memory space can be broken into small, unusable pieces, leading to wasted memory and reduced performance.

Complexity: Implementing and managing segmentation can be more complex than paging. Managing multiple segments per process is challenging, and the likelihood of segmentation faults can increase.